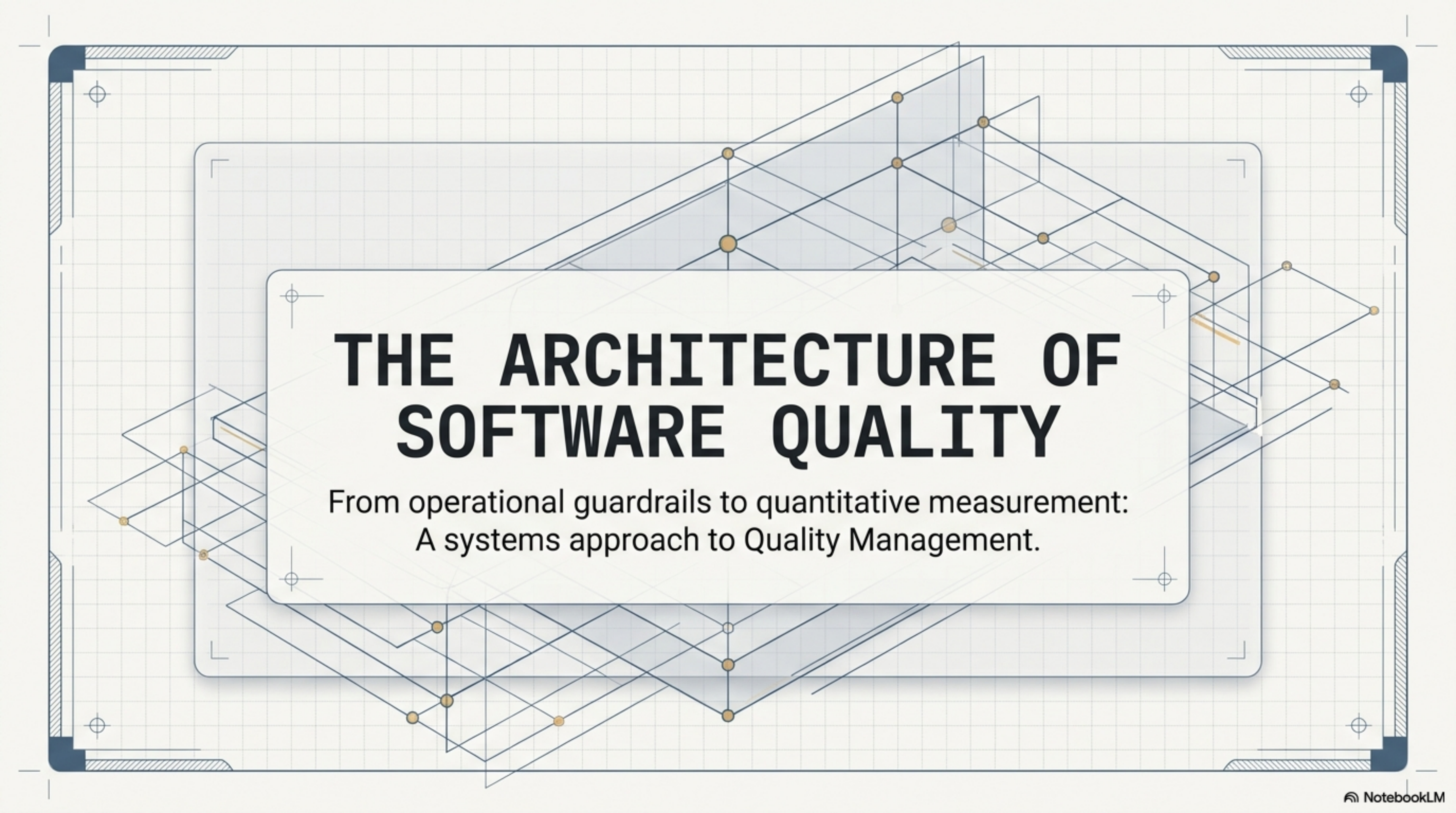
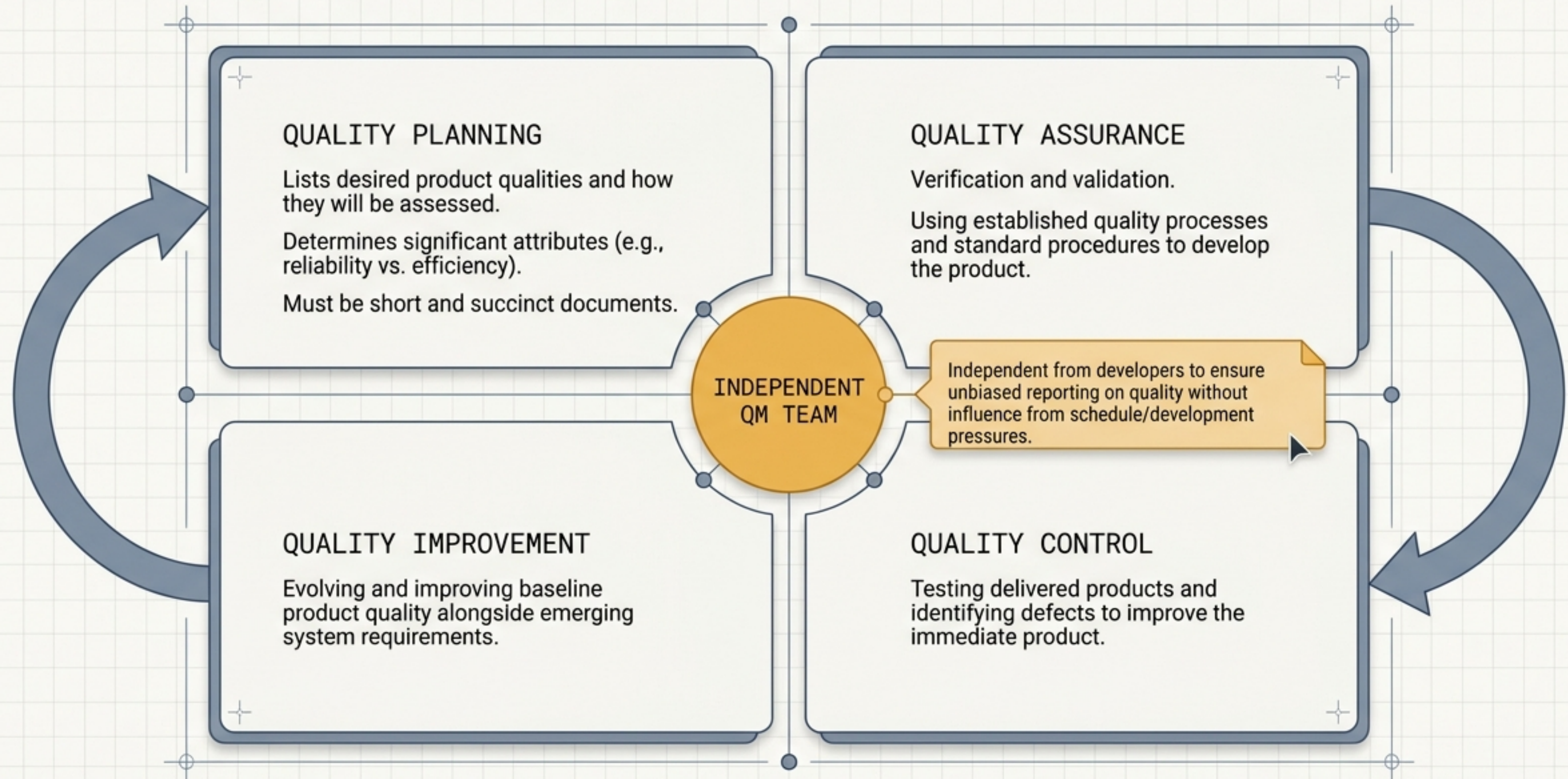




THE ARCHITECTURE OF SOFTWARE QUALITY

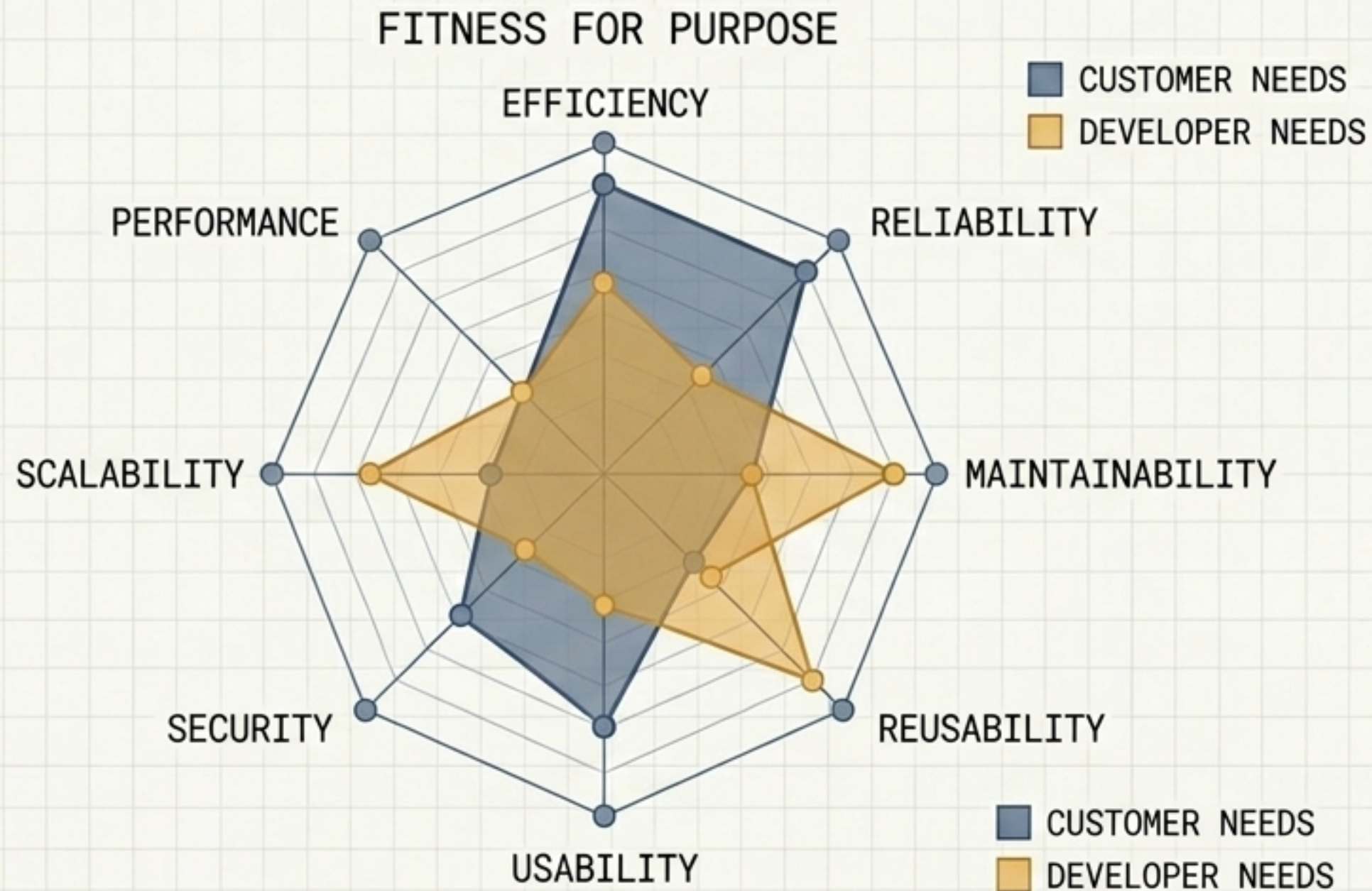
From operational guardrails to quantitative measurement:
A systems approach to Quality Management.

THE QUALITY MANAGEMENT ENGINE



THE SPECIFICATION FALLACY

Conformance to specifications is often impossible because specs are naturally ambiguous, incomplete, or inconsistent.

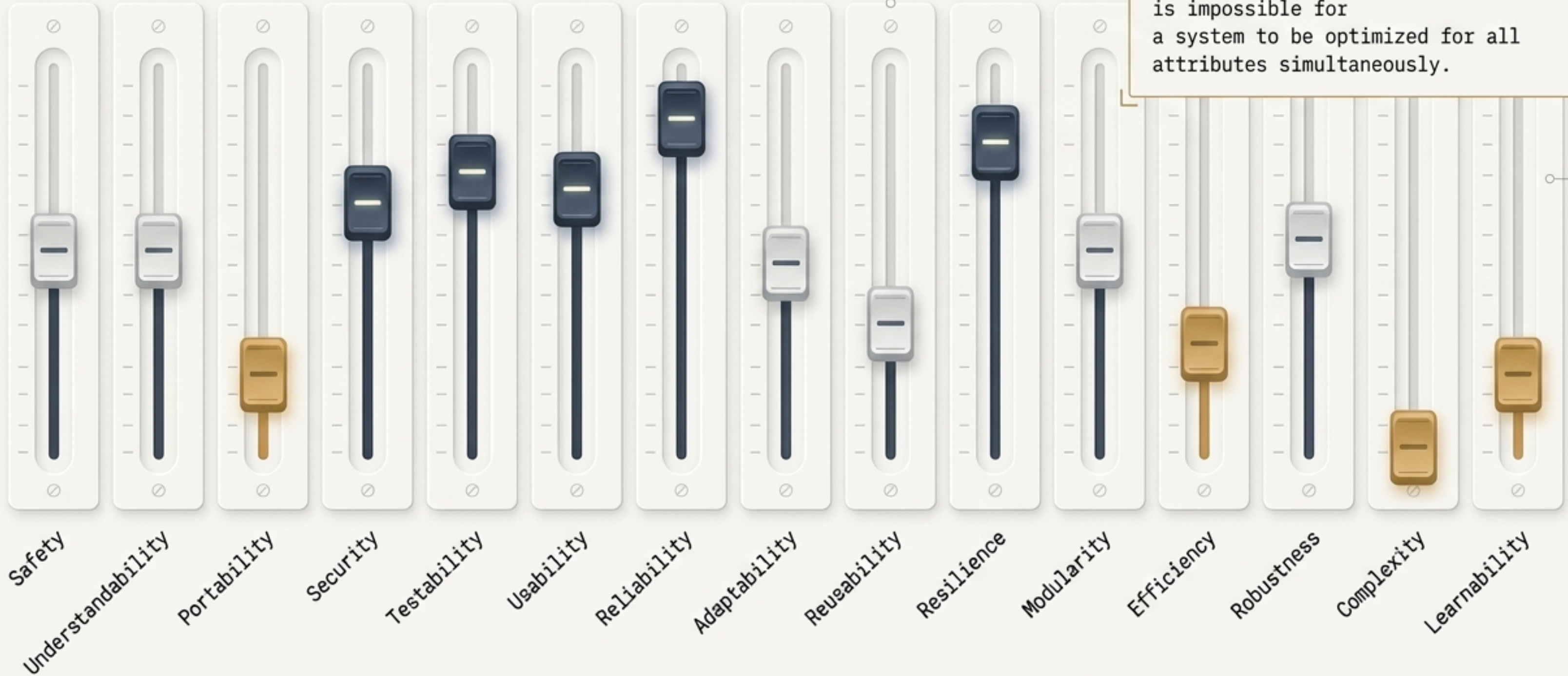


'FITNESS FOR PURPOSE' CHECKS:

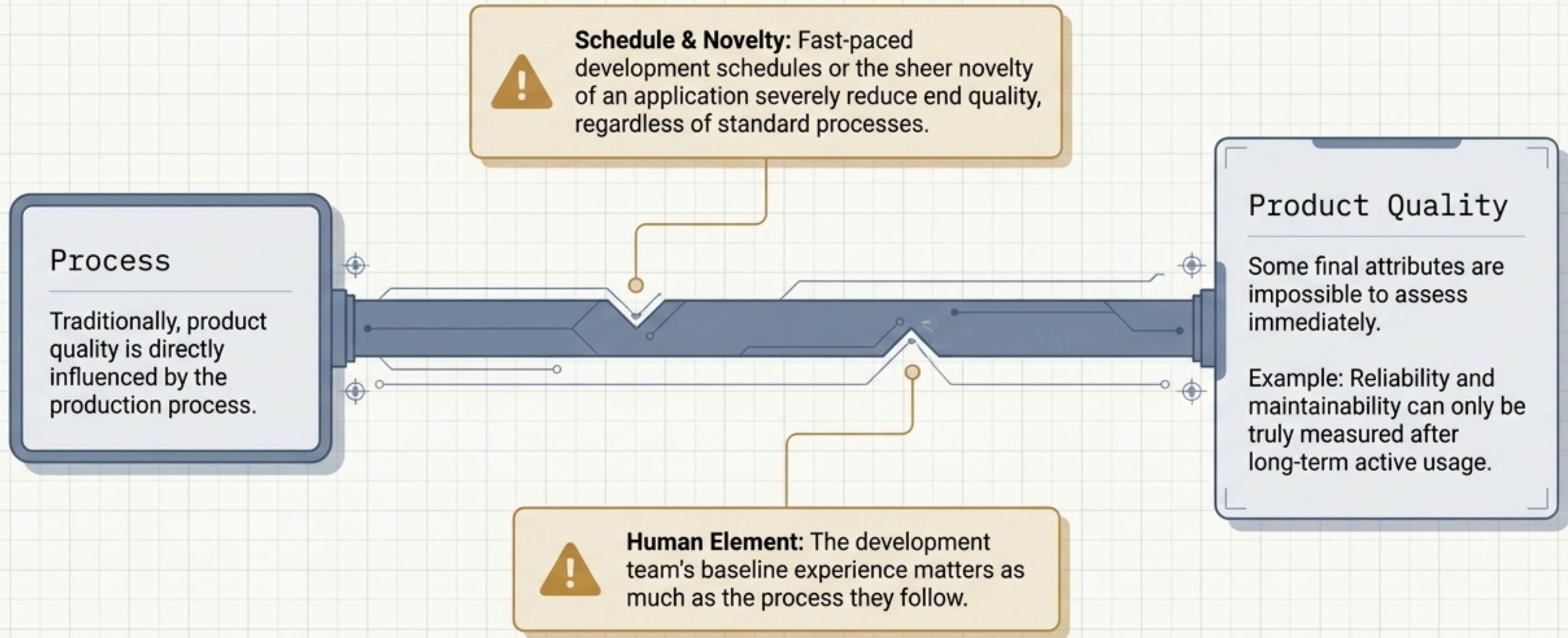
- ☒ - Has the software been properly tested?
- ☒ - Is performance acceptable for normal use?
- ☒ - Is it usable?
- ☒ - Is it well-structured and understandable?
- ☒ - Were programming and documentation standards followed?

Non-Functional Trade-offs

Practical quality assessment requires selecting and prioritizing a specific subset of these attributes based on the product's primary use case. It is impossible for a system to be optimized for all attributes simultaneously.



The Process-Product Disconnect



Operational Guardrails: Software Standards

The Function of Standards

Standards define exactly how software processes should be put into practice.

Why they matter:

- Codify organizational best practices
- Reduce learning curves for new staff
- Ensure work continuity across teams

External Standards:
International or national regulations.

Internal Standards:
Organizational or specific project standards.

System Warning: Vulnerabilities

Obsolescence: Fast-evolving technologies quickly outdate rigid standards; requires regular revision.

Overhead: Risk of excessive form-filling and documentation bloat.

Rigidity: Different software types require different development models; standards cannot be universally rigid.

Diagnostic Matrix: Product vs. Process Standards

Product Standards	Process Standards
(Applied directly to the end product)	(Applied to the underlying development activities)
Design review form	Design review conduct
Requirements document structure	Submission of new code for system building
Method header format	Version release process
Java programming style	Project plan approval process
Project plan format	Change control process
Change request form	Test recording process

Assurance vs. Control



Quality Assurance (QA)

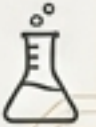
Focus:
The Process.

Action:
Checking documentation, the development process, and non-conformance with standards.

Nature:
Preventative. Using procedures to develop a quality product side-by-side with verification and validation.



Quality Control (QC)

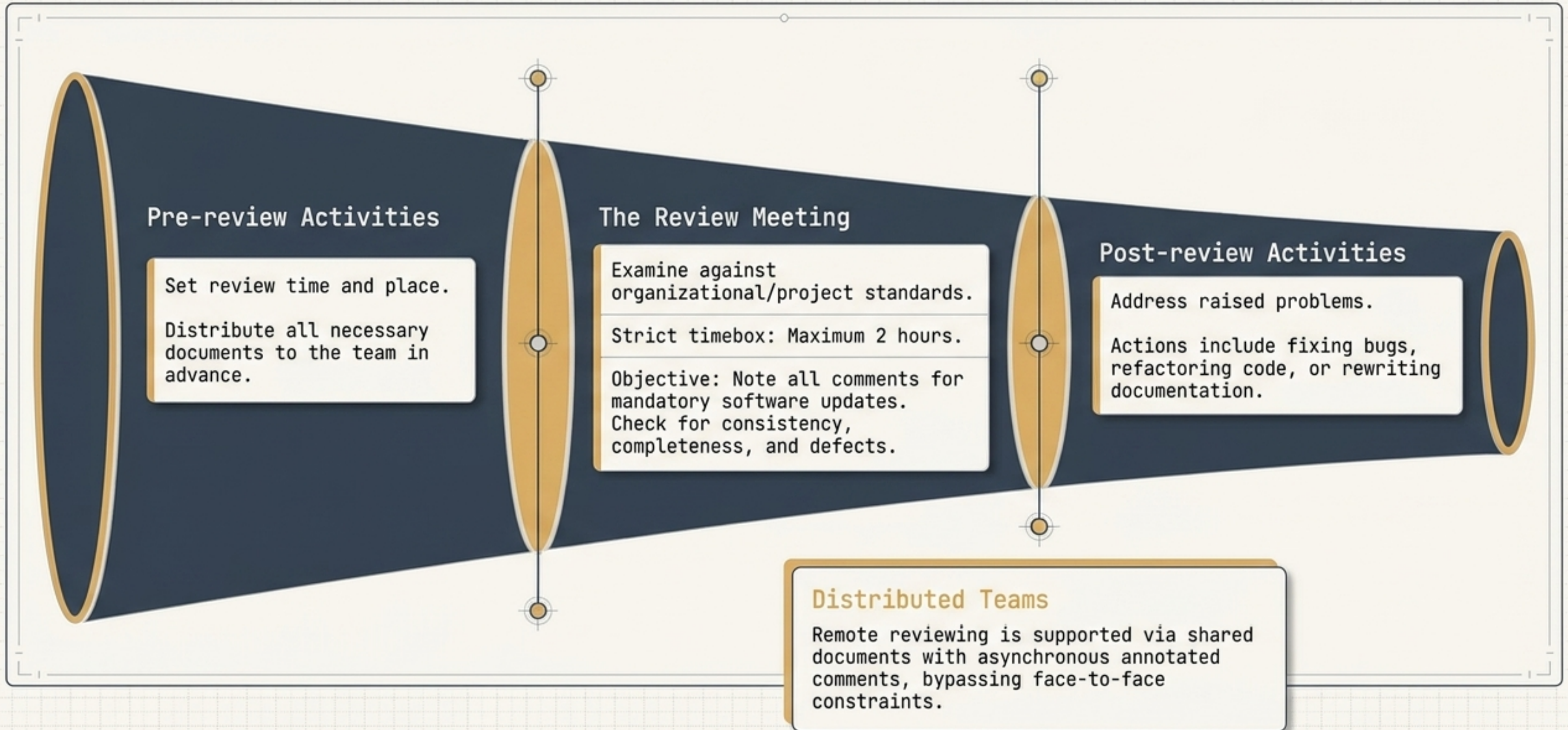


Focus:
The Product.

Action:
Testing the delivered software, checking the executable code.

Nature:
Reactive. Identifying specific defects to immediately improve the product.

The Review Pipeline



Anatomy of a Code Inspection

Inspections remove up to 60% of program defects prior to execution.

Data Faults

Are variables initialized before use? Is array upper bound Size or Size-1? Buffer overflow risks?

Interface & I/O

Correct parameter order/types? Do unexpected inputs corrupt the system? All output variables assigned?

```
1  //////////////////////////////////////
2  //FileName:      : Program.cs
3  //FileType:      : Visual C# Source file
4  //Size           : 2721
5  //Author         : Your Name
6  //Created On     : 8/6/2015 12:27:26 PM
7  //Last Modified On : 8/10/2015 3:38:03 PM
8  //Copy Rights    : Your company
9  //Description     : This is my test class
10 //////////////////////////////////////
11
12 using System;
13 using System.Collections.Generic;
14 using System.Linq;
15 using System.Text;
16 using System.IO;
17 ...
18 public method Teanen(Strin(strong, parameter: *args, Methor[] args)
19 {
20     if (altkbs = 0. memoryString <= null)
21     {
22         exception = ecepWrite();
23         memory.write();
24         oisero.tohandle();
25     }
26 }
```

Control Faults

Is loop termination certain? Compound statements bracketed? All switch cases accounted for?

Storage & Exceptions

Is dynamic space explicitly deallocated? Are all possible error conditions handled?

The Quantitative Dashboard: Process Metrics

Software measurement derives numeric values for attributes to allow objective comparison. Despite its power, systematic measurement is still underutilized in most organizations.

Time Metrics

Measurement of chronological duration.

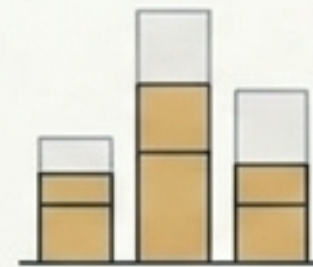


Example:

Total calendar time to complete a specific development phase.

Resource Metrics

Measurement of consumption.



Examples:

Total effort in person-days, required computer resources, and associated travel costs.

Event Occurrences

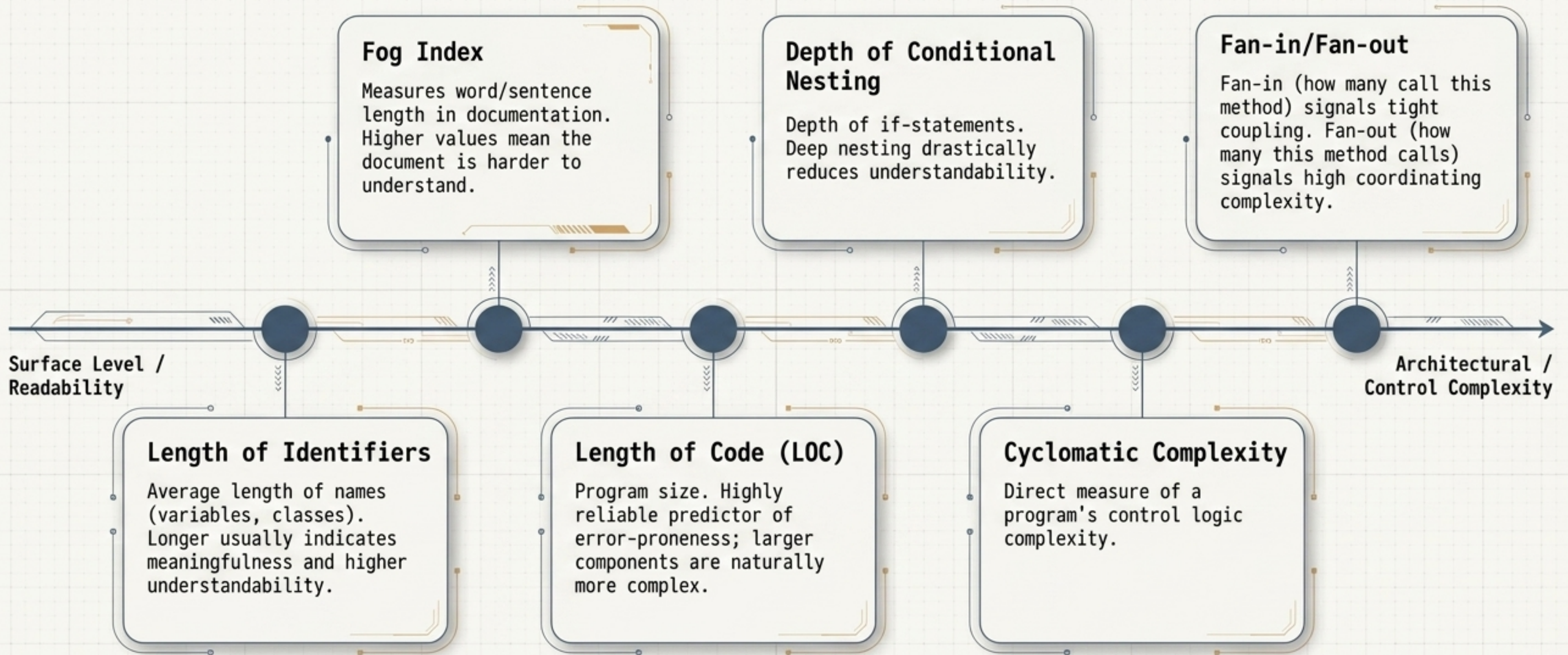
Measurement of frequency.



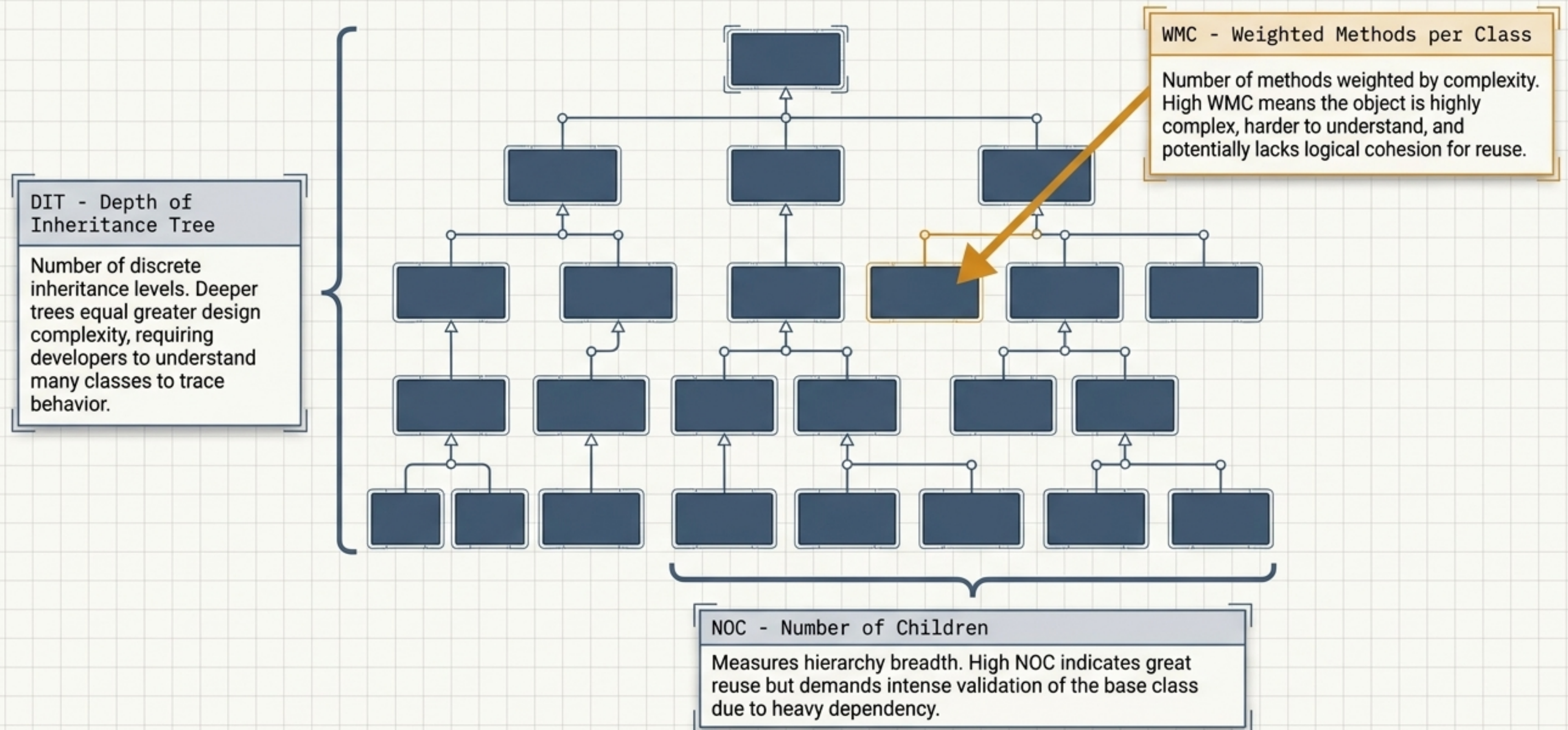
Examples:

Number of defects discovered during inspection, requirements changes requested, bug reports in delivered systems, average LOC modified per change.

The Static Product Metric Spectrum



Object-Oriented Measurement Topology



Synthesis: The Quality Feedback Engine



Software Quality Management is not a checklist;
it is an engineered, self-correcting system.